

Designing Classes

Access modifiers

Encapsulation, this

Inheritance

Function overloading & overriding



Class

```
class Car
{
    // member variables
    // member methods
}
```

Example:

```
class Car {    //these are member variables
    String manufacturer;
    String model;
    int year;
    int passengers;
}
```

Encapsulation

- One of advantages of OOP is encapsulation
- ability to define classes that **hide their impl. details**, exposing only public methods to other classes
- Support for encapsulation in Java comes from three keywords: **public**, **private**, and **protected**
- these access modifiers specify where a member is accessible
- **public** is entirely accessible to all other classes (whether in same package or outside)

Public access modifier

```
class Car // all are public
{
    public String manufacturer;
    public String model;
    public int year;
    public int passengers;
}
```

Private access modifier

- Declaring all members as public, does not achieve the goal of encapsulation since it permits direct access to the class members from outside the class
- To restrict access to a field, use the keyword **private**
- A private member is available only within the same class

Private access modifier cont.

Suppose the Car class is intended for use in a used car sales application. In this case, in order to hide the cost of car from potential buyers:

```
class Car {  
    public String manufacturer;  
    public String model;  
    public int year;  
    public int passengers;  
    private float cost; }
```

Protected access modifier

- members declared as **protected** are accessible within the same class and classes derived from the class, but not to other classes
- The ability to derive a class from another class will be discussed later

static member

- In addition to private, protected and public members, a Java class can also have static members
- A static member is often referred to as class members or class instances since they are accessed with reference to class name instead of instance (object) name.

Member Methods

- Similar to member variables, access to member methods can be controlled with the `public`, `protected` and `private` modifiers

```
[methodModifiers] returnType methodName  
    ([param1],[param2]...) [throws exceptionList]  
{    // method body  
}
```

Member Methods(modifiers)

Modifier	Purpose
Public	Accessible outside the class in which it is declared.
Protected	Accessible by the class in which it is declared and by subclasses of that class.
Private	Accessible only by the class in which it is declared.
Static	A method of the class rather than of a particular instance of the class.
Abstract	Not implemented in this class.
Final	Cannot be overridden in subclasses.
Native	A platform-dependent implementation of the method in another language, typically C or assembly.
Synchronized	Used to indicate a critical method, that will lock the object to prevent execution of other methods, while the synchronized method executes.

Return type of a method

- can be one of the primitive types (for example, int, float, char), another class, or void

Overloaded Methods

- Use the same method name for more than one methods

Example:

```
private float cost;
```

```
public float CalculateSalePrice() {  
    return cost * 1.5; } }
```

```
public float CalculateSalePrice(double margin) {  
    return cost * (1 + margin); } }
```

The **this** Variable

- The `this` can be used at runtime to reference the object itself

```
public class Car
{ String manufacturer; String model; int year; int passengers; float cost;
    // calculate the sale price of a car based on its cost
    public double CalculateSalePrice() {
        return cost * 1.5;    }
    // a public constructor
    public Car(String manufacturer, String model, int year, int passengers, float cost)
    { this.manufacturer = manufacturer;
      this.model = model;
      this.year = year;
      this.passengers = passengers;
      this.cost = cost;
    }
}
```

Class Inheritance

- Every class in Java is derived from another class
- A derived class is referred to as a **subclass**,
- while the class it is derived from is referred to as **superclass**.
- A class can be derived from another by using **extends** keyword as follows:

Example:

```
public class ClassicCar extends Car
{
    // member methods and variables
}
```

Extends by default

- A class is not declared as being derived from a specific class is implicitly derived from Java base class, **Object**.
- This means that the following two class declarations are same:

```
public class Car {  
    // member methods and variables  
}
```

```
public class Car extends Object {  
    // member methods and variables  
}
```

Overriding member methods

- While creating a subclass, all functionalities of its superclass are inherited
- The subclass can add or change this functionality as desired

```
public class Car {  
    private int year;  
    private float originalPrice;           // calculate the sale price of a car based on its cost  
    public double CalculateSalePrice() {  
        double salePrice;  
        if (year > 1994)                   salePrice = originalPrice * 0.75;  
        else if (year > 1990)              salePrice = originalPrice * 0.50;  
        else                               salePrice = originalPrice * 0.25;  
        return salePrice;  
    }                                     // a public constructor  
    public Car(int year, float originalPrice) {  
        this.year = year;  
        this.originalPrice = originalPrice;  
    }  
}
```


Subclass inherit car

```
public class ClassicCar extends Car {  
    // calculate the sale price of a car based on its cost  
    public double CalculateSalePrice() {  
        return 10000;  
    }  
  
    // a public constructor  
    public ClassicCar(int year, float originalPrice) {  
        super(year, originalPrice);  
    }  
}
```

Cont.

```
ClassicCar myClassic = new ClassicCar(1920, 1400);  
double classicPrice = myClassic.CalculateSalePrice();
```

```
Car myCar = new Car(1990, 12000);  
double price = myCar.CalculateSalePrice();
```

- Variable myClassic is of type ClassicCar. Because myClassic is a ClassicCar, the sale price will be \$10,000.
- Next, myCar is constructed as a new object of type Car. Because myCar is a Car, its sale price will be based on the year it was made (1990) and will be 25% of \$12,000, or \$3,000.